

Défrichage

Chat Gpt entrées :

- Je suis un étudiant en première année de licence et je découvre le C. Je veux que tu me fasses un petit cours sur les pointeurs en C. Jusqu'à là je n'ai vu que les variables, les conditions, les boucles et les fonctions. Je ne veux pas l'allocation dynamique ni la passage par référence.

Rappel : utiliser la mémoire

Dans un programme on a besoin de :

- Stocker les valeurs dans la mémoire.
- Puis ensuite les lire, les remplacer ou les effacer.

Déclarer une variable : allouer un espace mémoire et le manipuler

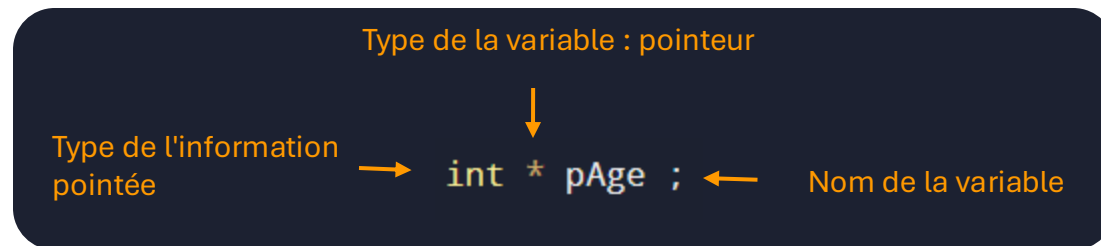
Adresse (numéro de la case)	0	1		2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
Valeur stockée	0	1	1	0	1	1	1	0	0	1	0	0	1	1	1	1	1	1	1	1	0	0	0	1	0	0	1						

Mémoire

Variable : un nom, un type, une valeur , une adresse de départ

Mais on peut aussi utiliser uniquement l'adresse et le type : sans passer par une variable.

Un pointeur stocke une adresse mémoire et le type d'information qui s'y trouve.



Conventions :

- Le nom démarre par p
- * collée au nom de la variable

```
int *pVegetablePrice;
```

Accéder à l'adresse d'une variable : &

Récupérer l'adresse d'une variable : & devant

```
int a = 3;  
printf("%d\n",a);  
printf("%p\n",&a);
```

→ %p pour les adresses (pointeurs)

Lancement 1 :	3 0x7ffe193ac17c
Lancement 2:	3 0x7ffcd50979ec
Lancement 3 :	3 0x7ffced2688fc

Stocker l'adresse dans un pointeur :

```
pAge = &a ;
```

En mémoire que se passe-t-il ?

```
int a = 3;  
int *pAge = &a;  
printf("%d\n", a);  
printf("%p\n", pAge);
```

→ 3
0x7ffcc

Adresse
(numéro de la case)

Valeur stockée

... 0x7ffcc ... 0x9ffd ...

... 3 ... 0x7ffcc ...

a

pAge

Comment accéder à l'information pointée : *

```
int a = 3;
int *pAge = &a;
printf("%d\n", a );
printf("%p\n", pAge );
printf("%d\n", *pAge );
```

→ 3
0x7ffcc
3

affiche la valeur contenu dans
l'adresse mémoire dans ce contexte

Adresse
(numéro de la case)

Valeur stockée

... 0x7ffcc ... 0x9ffd ...

...	3	...	0x7ffcc	...
-----	---	-----	---------	-----

a

pAge

... 0x7ffcc ... 0x9ffd ...

...	6	...	0x7ffcc	...
-----	---	-----	---------	-----

a

pAge

... 0x7ffcc ... 0x9ffd ...

...	9	...	0x7ffcc	...
-----	---	-----	---------	-----

a

pAge

```
int a = 3;
int *pAge = &a;
printf("%d\n", a );
*pAge = 6;
printf("%d\n", *pAge );
printf("%d\n", a );
```

→ 3
6
6

```
int a = 3;
int *pAge = &a;
printf("%d\n", a );
*pAge = 6 + *pAge;
printf("%d\n", *pAge );
printf("%d\n", a );
```

→ 3
9
9

Ne pas confondre

à la déclaration

```
int *pAge;
```

Indication du type de la variable :
pointeur

à l'utilisation

```
if ( *pAge >= 18 )
```

Permet d'accéder à la valeur pointée
(au lieu d'accéder à l'adresse)

Erreurs courantes

Null Pointer Exception

Le pointeur vaut NULL

Pas d'initialisation

ou

Mise à NULL volontaire

ou

Désallocation dynamique
(voir CM6)

Invalid Pointer Error

Le pointeur contient une adresse vers
un espace non alloué.

L'adresse a été modifiée
(au lieu de la valeur par exemple)

ou

L'espace a été désalloué

Ex :

```
int *pAge ;
{
    int a = 3;
    pAge = &a;
}
printf("%d\n", *pAge );
```



Application dans les fonctions

Mettre des pointeurs en paramètres pour contourner la limite d'un seul élément retourné.

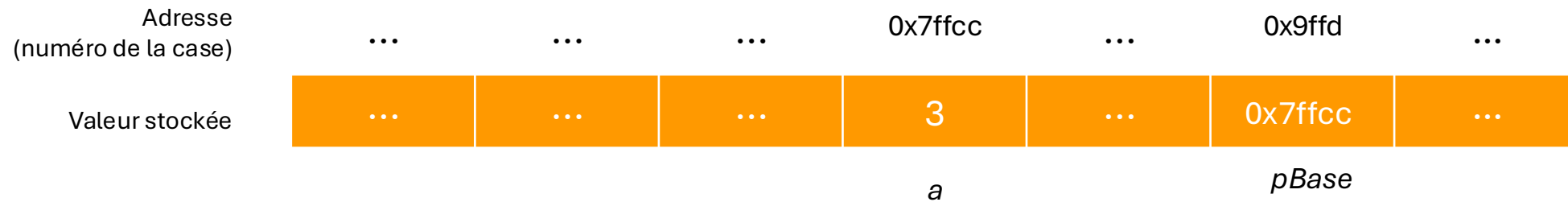
	in	out (1)	in-out	out (x)
C	Paramètre	return	Pointeurs	
C++	Paramètre	return		

Paramètres in-out

Le pointeur permet de modifier l'espace mémoire initial (et donc de faire une sorte de retour).

```
int main()
{
    int a = 3;
    addOne ( &a );
    printf("%d\n", a );
}
```

```
void addOne (int *pBase)
{
    *pBase = *pBase + 1;
}
```



Application dans les fonctions

Toujours un passage par valeur

```
int main()
{
    int a;
    int *pDay =&a;
    getBirthDate ( pDay );
    printf("%d\n", a );
    printf("%d\n", *pDay );
}
```

```
void getBirthDate ( int *pDay)
{
    scanf("%d", *pDay);
}
```

Adresse
(numéro de la case)

... 0x7fcc ... 0x12e56 ... 0x9ffd ...

Valeur stockée

... 0x12e56 ... 15 ... 0x12e56 ...

pDay
(main)*a*pDay
(fonction)

Pointeur de pointeur

Possible de pointer vers un pointeur.

```
int* *pPInfo;
```

```
int a = 42;  
int *pB = &a;  
int* *pPC = &pB;
```

```
printf("%p\n", pPC);  
printf("%p\n", *pPC);  
printf("%d\n", **pPC);
```

← L'adresse stockée dans pPC (donc l'adresse de pB)

← L'adresse stockée dans pB (donc l'adresse de a)

← La valeur stockée dans a (donc 42)

Adresse
(numéro de la case)

Valeur stockée

...	0x7ffcc	...	0x12e56	...	0x9ffd	...
...	42	...	0x7ffcc	...	0x12e56	...
	a		pB		pPC	